possible to track memory allocations in the type system if that really mattered to us. But in Scala we have the benefit of automatic memory management, so the cost of explicit tracking is usually higher than the benefit.

The policy we should adopt is to *track those effects that program correctness depends on.* If a program is fundamentally about reading and writing files, then file I/O should be tracked in the type system to the extent feasible. If a program relies on object reference equality, it would be nice to know that statically as well. Static type information lets us know what kinds of effects are involved, and thereby lets us make educated decisions about whether they matter to us in a given context.

The `ST` type in this chapter and the `IO` monad in the previous chapter should have given you a taste for what it's like to track effects in the type system. But this isn't the end of the road. You're limited only by your imagination and the expressiveness of Scala's types.

14.4 Summary

In this chapter, we discussed two different implications of referential transparency.

We saw that we can get away with mutating data that never escapes a local scope. At first blush it may seem that mutating state can't be compatible with pure functions. But as we've seen, we can write components that have a pure interface and mutate local state behind the scenes, and we can use Scala's type system to guarantee purity.

We also discussed that what counts as a side effect is actually a choice made by the programmer or language designer. When we talk about functions being pure, we should have already chosen a context that establishes what it means for two things to be equal, what it means to execute a program, and which effects we care to take into account when assigning meaning to our program.